National Institute Of Technology Agartala

PROJECT

Automatic Transformer Protection System

prepared by

KUNGA CHUSKIT
Electrical Engineering

1. INTRODUCTION

Transformers are essential components of electrical power systems, as they enable the transfer of electrical energy between circuits at different voltage levels. They are widely used in power generation, transmission, distribution, and industrial applications. However, transformers may experience abnormal operating conditions such as overvoltage, undervoltage, overcurrent, and excessive temperature, which can affect their performance, reduce their service life, and potentially lead to equipment damage. Therefore, an effective protection system is necessary to ensure the safe and reliable operation of transformers.

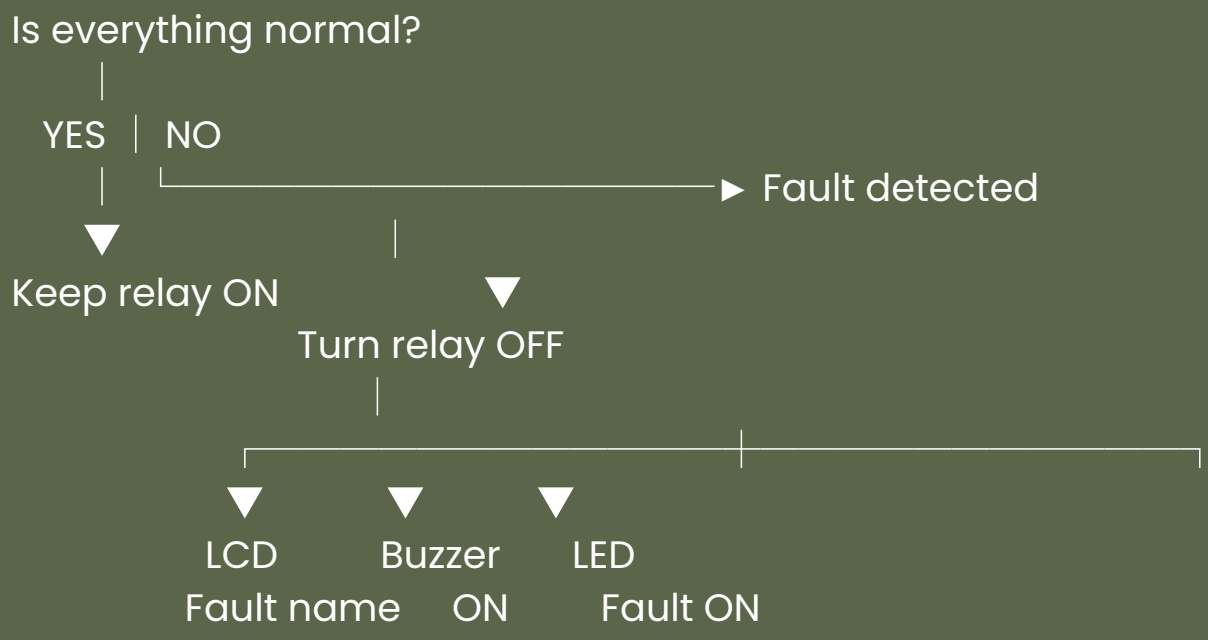
The Automatic Transformer Protection and Monitoring System is designed to detect abnormal operating conditions and provide automatic protection against potential faults. The system uses an Arduino Uno microcontroller to monitor important electrical and thermal parameters of a transformer through suitable sensors. A voltage sensor, current sensor, and temperature sensor are used to measure the transformer's operating voltage, current, and temperature, respectively.

The measured parameters are continuously processed by the microcontroller and compared with predefined safety limits. When an abnormal condition is detected, the system activates a relay to disconnect the load, thereby helping to prevent further damage. An LCD display, LEDs, and a buzzer provide real-time information about the operating condition and indicate the occurrence of a fault. A manual reset mechanism is also incorporated to allow the system to resume operation after the fault condition has been addressed.

The main objective of this project is to develop a simple, cost-effective, and microcontroller-based transformer protection and monitoring system. The project demonstrates how sensors, embedded systems, and relay-based switching can be integrated to improve transformer safety and operational reliability. The developed prototype provides a basic demonstration of automatic protection principles and can serve as a foundation for understanding more advanced transformer protection systems used in industrial power networks.

1. Voltage sensor
Measures transformer voltage.
Arduino checks whether it is too high or too low.
Too high → Overvoltage fault
Too low → Undervoltage fault
2. Current sensor
Measures load/transformer current.
If current exceeds the preset limit → Overcurrent fault.
3. Temperature sensor
Measures transformer temperature.
If temperature exceeds the preset limit → Overtemperature fault.
All these sensor signals go to the Arduino.

The Arduino essentially makes this decision:



The four protection conditions

Fault	What Arduino detects	Action
Overvoltage	V > Vmax	Trip relay
Undervoltage	V < Vmin	Trip relay
Overcurrent	I > Imax	Trip relay
Overtemperature	T > Tmax	Trip relay

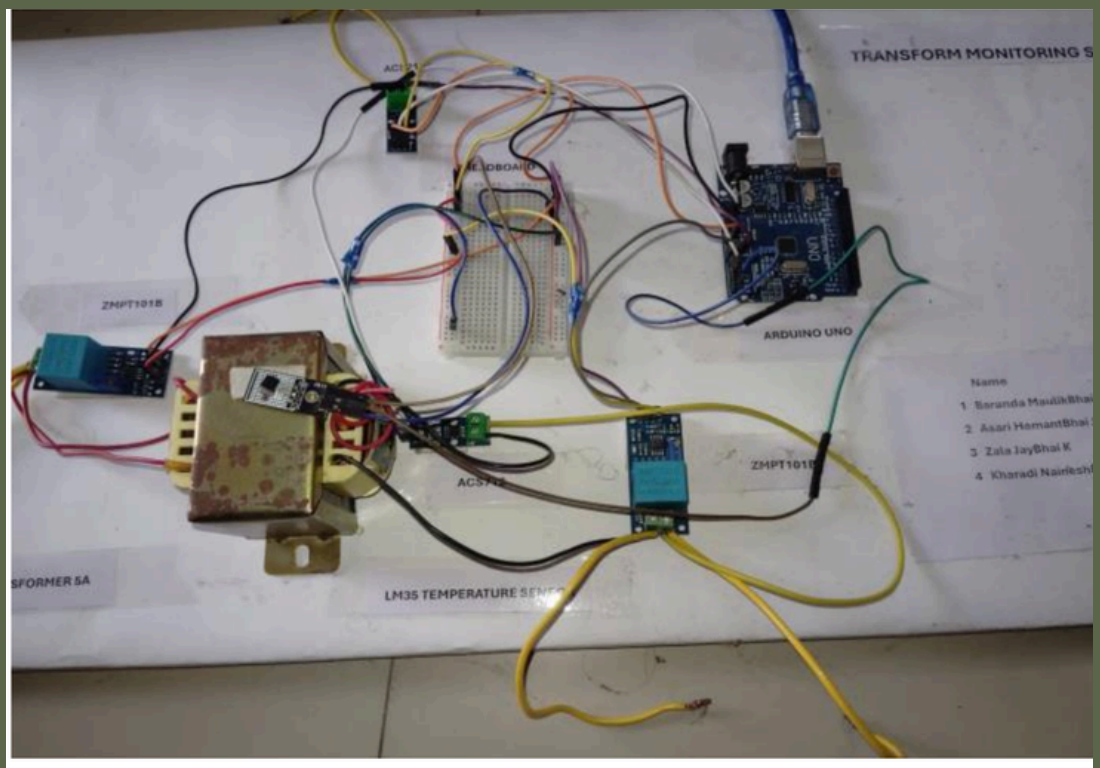


Figure 1: Prototype of Proposed system

1. Arduino Uno — the brain
The Arduino continuously reads:
Voltage → Current → Temperature and compares them with predefined limits.

2. Voltage Sensor
The voltage sensor converts the transformer voltage into a safe, measurable signal for Arduino.
The Arduino then determines:
V > Vmax → Overvoltage
V < Vmin → Undervoltage
otherwise → Normal
Interview point: Arduino cannot directly measure a high AC voltage. We need a properly designed/rated sensing and isolation interface to scale/isolate it to a safe level.

3. ACS712 Current Sensor
This measures the current flowing to the load.
The ACS712 produces an electrical signal proportional to the current.
Arduino reads that signal and calculates/estimates the current.
I > Imax → Overcurrent → Trip
For example, if the allowed current is 2 A and the measured current becomes 2.5 A, Arduino trips the relay.

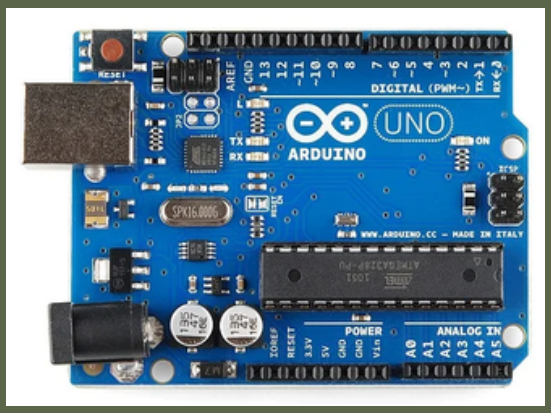
4. LM35 Temperature Sensor
The LM35 gives an analog output proportional to temperature.
Approximately:
10 mV/°C
So:
25°C → ~250 mV
50°C → ~500 mV
70°C → ~700 mV
Arduino reads this voltage and converts it into temperature.
If:
T > Tmax
→ Overtemperature fault

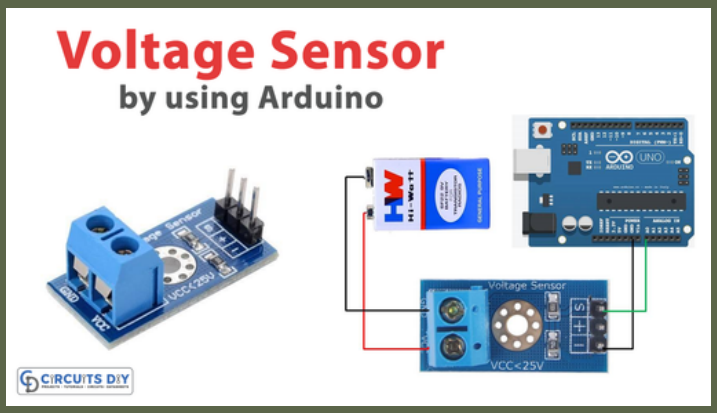
5. Relay
This is extremely important.
The Arduino does not directly disconnect transformer/load.
Instead:
Arduino → Relay driver → Relay contacts → Load
When a fault is detected, Arduino changes the relay state and the relay opens the power circuit, disconnecting the load.

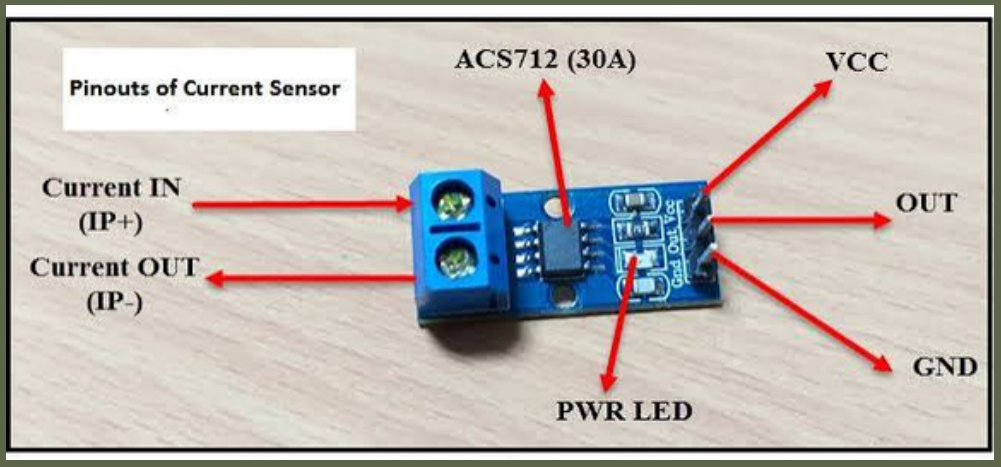
6. LCD
The LCD can show something like:
V: 230V I: 1.2A
T: 45C NORMAL
During a fault:
V: 270V
OVERVOLTAGE!
This makes the system easy to monitor.

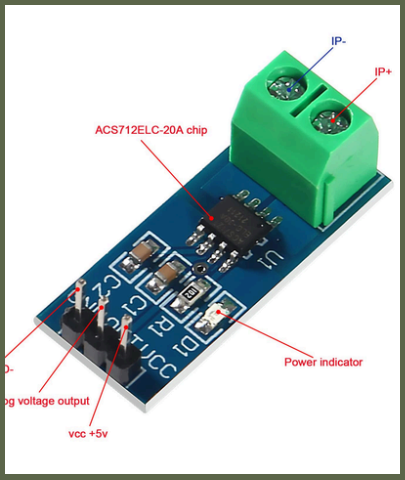
8. Push Button — Manual Reset
Suppose an overcurrent occurs.
The system trips:
Fault detected
↓
Relay OFF
↓
Load disconnected
After fixing the problem, the operator presses the RESET button.
Arduino then checks the conditions again.

Equipments









Device	Arduino Pin
ZMPT101B OUT	**A0**
ACS712 OUT	**A1**
LM35 OUT	**A2**
Relay IN	**D8**
Buzzer	**D9**
Reset button	**D10**
LCD RS	**D12**
LCD E	**D11**
LCD D4	**D5**
LCD D5	**D4**
LCD D6	**D3**
LCD D7	**D2**
Green LED	**D6**
Red LED	**D7**

FAULT

↓

Relay OFF

↓

Load disconnected

↓

Buzzer ON

↓

Red LED ON

↓

LCD displays fault

Press RESET

↓

Arduino checks conditions

↓

Are all parameters normal?

↓

YES ——→ Relay ON

1. LM35 calibration — Temperature
This is straightforward.
The LM35 output is approximately:
Arduino ADC:
Then:
For example, if Arduino reads:
then:
Therefore:
T≈49.9°C

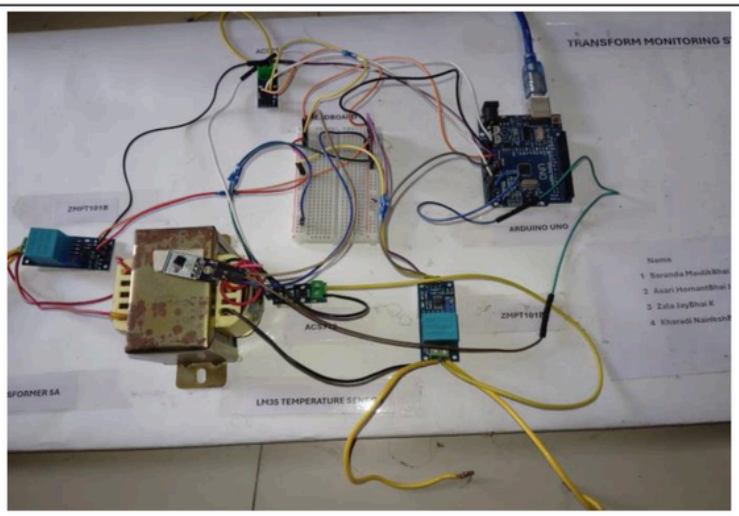


Figure 1: Prototype of Proposed system

```
#include <LiquidCrystal.h>

// ---------- PIN DEFINITIONS ----------

#define VOLTAGE_PIN A0
#define CURRENT_PIN A1
#define TEMP_PIN    A2

#define RELAY_PIN   8
#define BUZZER_PIN  9
#define RESET_PIN   10

#define GREEN_LED   6
#define RED_LED     7

// LCD: RS, E, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);


// ---------- PROTECTION LIMITS ----------

float Vmin = 180.0;
float Vmax = 250.0;

float Imax = 2.0;

float Tmax = 70.0;


// ---------- SYSTEM STATE ----------

bool faultState = false;
String faultType = "";


// ---------- SETUP ----------

void setup()
{
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

  pinMode(RESET_PIN, INPUT_PULLUP);

  lcd.begin(16, 2);

  // Initially system is normal
  digitalWrite(RELAY_PIN, HIGH);
  digitalWrite(BUZZER_PIN, LOW);

  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);

  lcd.print("Transformer");
  lcd.setCursor(0, 1);
  lcd.print("Protection");
  delay(2000);

  lcd.clear();
}


// ---------- MAIN LOOP ----------

void loop()
{
  // Read sensor values
  float voltage = readVoltage();
  float current = readCurrent();
  float temperature = readTemperature();


  // If system is already tripped,
  // keep relay OFF until manual reset
  if (faultState == true)
  {
    digitalWrite(RELAY_PIN, LOW);
    digitalWrite(BUZZER_PIN, HIGH);

    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);

    displayFault();

    // Check reset button
    if (digitalRead(RESET_PIN) == LOW)
    {
      delay(50);

      if (digitalRead(RESET_PIN) == LOW)
      {
        // Only reset if conditions are safe
        if (voltage >= Vmin &&
            voltage <= Vmax &&
            current <= Imax &&
            temperature <= Tmax)
        {
          faultState = false;
          faultType = "";

          digitalWrite(RELAY_PIN, HIGH);
          digitalWrite(BUZZER_PIN, LOW);

          digitalWrite(GREEN_LED, HIGH);
          digitalWrite(RED_LED, LOW);

          lcd.clear();
          lcd.print("System Reset");
          delay(1000);
        }
      }
    }
```

```
  return;
 }


 // ---------------- FAULT DETECTION ----------------

 if (voltage > Vmax)
 {
  tripSystem("OVERVOLTAGE");
 }

 else if (voltage < Vmin)
 {
  tripSystem("UNDERVOLTAGE");
 }

 else if (current > Imax)
 {
  tripSystem("OVERCURRENT");
 }

 else if (temperature > Tmax)
 {
  tripSystem("OVER TEMP");
 }

 else
 {
  // Everything normal
  digitalWrite(RELAY_PIN, HIGH);
  digitalWrite(BUZZER_PIN, LOW);

  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);

  displayNormal(voltage, current, temperature);
 }

 delay(200);
}


// ---------------- TEMPERATURE ----------------

float readTemperature()
{
 int adcValue = analogRead(TEMP_PIN);

 float voltage = adcValue * (5.0 / 1023.0);

 float temperature = voltage * 100.0;

 return temperature;
}
```

```
// ---------------- CURRENT ----------------

float readCurrent()
{
 int adcValue = analogRead(CURRENT_PIN);

 float voltage = adcValue * (5.0 / 1023.0);

 // These values MUST be calibrated
 float zeroVoltage = 2.5;
 float sensitivity = 0.185;   // ACS712-5A example

 float current =
    (voltage - zeroVoltage) / sensitivity;

 if (current < 0)
  current = -current;

 return current;
}


// ---------------- VOLTAGE ----------------

float readVoltage()
{
 /*
   The ZMPT101B produces an AC waveform.

   In the final hardware implementation,
   sample the waveform and calculate RMS.

   The calibration factor must be obtained
   experimentally from a known reference voltage.
 */

 float rmsVoltage = calculateZMPT_RMS();

 return rmsVoltage;
}


// Placeholder for ZMPT101B RMS calculation
float calculateZMPT_RMS()
{
 // Actual RMS sampling/calibration code
 // will depend on the ZMPT101B module setup.

 return 230.0;  // placeholder only
}


// ---------------- TRIP SYSTEM ----------------

void tripSystem(String fault)
{
 faultState = true;
 faultType = fault;

 digitalWrite(RELAY_PIN, LOW);
```

```
  digitalWrite(BUZZER_PIN, HIGH);

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, HIGH);

  lcd.clear();
  lcd.print("FAULT:");
  lcd.setCursor(0, 1);
  lcd.print(fault);
}


// ---------------- NORMAL DISPLAY ----------------

void displayNormal(float V, float I, float T)
{
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("V:");
  lcd.print(V, 1);

  lcd.print(" I:");
  lcd.print(I, 1);

  lcd.setCursor(0, 1);
  lcd.print("T:");
  lcd.print(T, 1);

  lcd.print("C OK");
}


// ---------------- FAULT DISPLAY ----------------

void displayFault()
{
  lcd.clear();

  lcd.print("FAULT:");

  lcd.setCursor(0, 1);
  lcd.print(faultType);
}
```

CONCLUSION

My project was an Arduino-based Automatic Transformer Protection System designed to protect the transformer and connected load from abnormal operating conditions. We monitored four parameters: voltage, current and temperature. For voltage measurement, we used a ZMPT101B voltage-sensing module, and for current measurement, we used an ACS712 current sensor. An LM35 temperature sensor was used to monitor the transformer temperature.

The sensor outputs were connected to the Arduino Uno, which acts as the main controller. The Arduino processes the sensor signals, calculates the corresponding voltage, current and temperature values, and compares them with predefined safe limits. If the voltage exceeds the upper limit, an overvoltage fault is detected; if it falls below the lower limit, an undervoltage fault is detected. Similarly, excessive current produces an overcurrent fault, and excessive temperature produces an overtemperature fault.

When any fault is detected, the Arduino de-energizes a relay to disconnect the load, while a buzzer and red LED provide an alarm and the LCD displays the fault condition. The fault is latched so that the load does not automatically reconnect. After the abnormal condition is cleared, the operator can press a manual reset button, and the Arduino first verifies that all parameters are within safe limits before reconnecting the load.

The main principle of the project is: sensing, processing, fault detection, relay isolation and indication.